



UNIVERSITY OF MORATUWA

Faculty of Information Technology

B.Sc. (Hons) ITM

Level 2 Semester 2

IN2410 – Database Systems

Practical 3 – Views in SQL

In SQL, a view is a virtual table based on the result-set of an SQL statement. A view, like a real table, will have rows and columns in its structure. The columns that make up a view can be actual columns from one or more real tables in the database, or they might be computed columns based on columns from those real tables. You can add SQL functions, WHERE, and JOIN statements to a view and present the data as if the data were coming from a single table.

Advantages of database view

The following are the advantages of using database views.

- A database view allows you to simplify complex queries: a database view is defined by an SQL statement that associates with many underlying tables. You can use database view to hide the complexity of underlying tables to the end-users and external applications. Through a database view, you only have to use simple SQL statements instead of complex ones with many joins.
- A database view helps limit data access to specific users. You may not want a subset of sensitive data to be queried by all users. You can use a database view to expose only non-sensitive data to a specific group of users.
- A database view provides an extra security layer. Security is a vital part of any relational database management system. The database view offers additional protection for a database management system. The database view allows you to create the read-only view to expose read-only data to specific users. Users can only retrieve data in read-only view but cannot update it.

- A database view enables computed columns. A database table should not have calculated columns however a database view should. Suppose in the *orderDetails* table you have *quantityOrder*(the number of ordered products) and *priceEach* (price per product item) columns. However, the *orderDetails* table does not have a computed column to store total sales for each line item of the order. If it has, the database schema would not be a good design. In this case, you can generate a column named *total* , which is a product of *quantityOrder* and *priceEach* to represent the calculated result. When you query data from the database view, the data of the computed column is calculated on the fly.
- A database view enables backward compatibility. Suppose you have a central database, which many applications are using it. One day, you decide to redesign the database to adapt to the new business requirements. You remove some tables and create new tables, and you don't want the changes to affect other applications. In this scenario, you can create database views with the same schema as the legacy tables that you will remove.

Disadvantages of database view

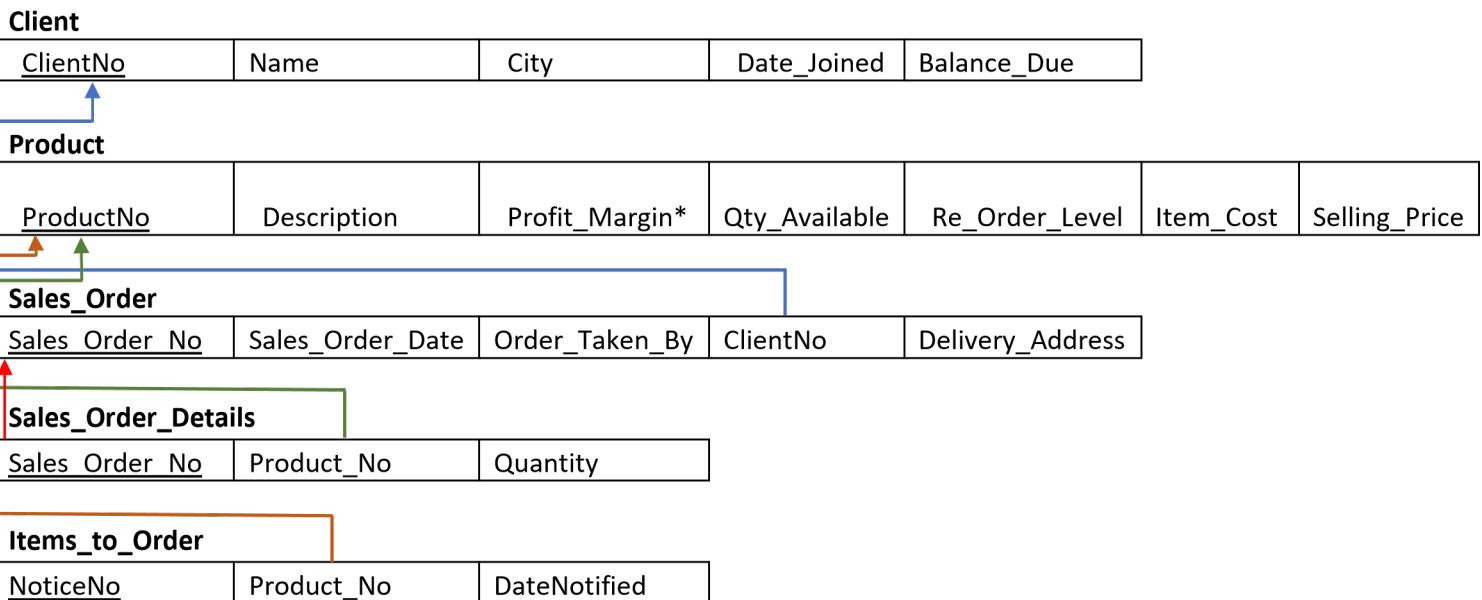
Besides the advantages above, there are several disadvantages of using database views:

- Performance: querying data from a database view can be slow, especially if the view is created based on other views.
- Tables dependency: you create a view based on underlying tables of the database. Whenever you change the structure of these tables that the view is associated with, you have to change the view as well.

Working Platform: Microsoft SQL Server

Note: Maximum time expected to spend on each question is mentioned within brackets. Try to finish within the time indicated.

1. Create the following tables under your database created during lab session 02. You may use the design editor to create them. Key fields are underlined. (5 mins)



2. Populate tables with appropriate records (Make sure to insert records in order to keep referential integrity). (10 mins)
3. Create views for the following. (45 mins)
 - 3.1. Prepare a list of sales orders placed by the client C005.
 - 3.2. Print the description and total quantity sold from each product
 - 3.3. Find the names of sales persons who have sold **HardDisk I TB**
 - 3.4. Display the sales order no and the day the customer placed their order
 - 3.5. Display the orders to be delivered in a given day
 - 3.6. Find the bill value of a given Sales_Order

4. Update Views. Discuss the following statements after executing them (30 mins)

4.1

(Note: there should be records with the date joined as '2016-05-20' in the Client table and Delivery_Addresses in Colombo for some clients joined in '2016-05-20')

```
CREATE VIEW view_1
AS
SELECT * FROM Sales_order WHERE ClientNo IN
(SELECT ClientNo FROM client WHERE Date_Joined='2016-05-20')
```

Then execute

```
UPDATE view_1 SET Delivery_Address = '58, Main Road, Colombo 8' WHERE
Delivery_Address LIKE '%Colombo%'
```

4.2

```
CREATE VIEW view_2
AS
SELECT product.ProductNo, product.Selling_Price, product.Selling_Price*
Sales_Order_Details.Quantity as Billvalue FROM
Sales_Order_Details, product
WHERE Sales_Order_Details.Product_No = Product.ProductNo AND Selling_Price* Quantity >1000;
```

Then execute

(Note: there should be records with selling price = 40)

```
UPDATE view_2 SET Selling_Price = 25 WHERE Selling_Price = 40;
```

Also, try (Note: there should be records with Product No 2)

```
UPDATE view_2 SET Selling_Price = 30 WHERE ProductNo = 'p0002';
```

4.3

(Note: there should be records with Profit margin – 10, 20, 50 in Product table)

```
CREATE VIEW view_3
AS
SELECT * FROM Product WHERE Profit_Margin>10;
```

Then execute (Note: there should be records with Profit margin above 50 in Product table)

```
UPDATE view_3 SET Qty_Available = 1500 WHERE Profit_Margin >50;
```

4.4

```
CREATE view Sales_Details
AS
SELECT so.Sales_Order_No,so.ClientNo,sod.Product_No,sod.Quantity
FROM Sales_Order so, Sales_Order_Details sod
WHERE so.Sales_Order_No=sod.Sales_Order_No
```

Then Execute

```
INSERT INTO Sales_Details (Sales_Order_No,ClientNo) VALUES(10,'C008')
```

4.5

```
CREATE view mytestview1
AS
SELECT c.clientNo,c.name,c.city,s.Sales_Order_No,s.Sales_Order_Date,s.Delivery_Address
FROM client c, Sales_Order s
WHERE c.clientNo=s.ClientNo
```

-- Update multiple base tables via views.

-- An important thing to remember is that SQL Server doesn't support updating multiple base tables at once via views.

```
UPDATE mytestview1
SET NAME='saman', Delivery_Address='Moratuwa'
WHERE clientNo='C001'
```

-- Update single base table via views.

```
UPDATE mytestview1
SET Delivery_Address='Moratuwa'
WHERE clientNo='C001'
```

```
UPDATE mytestview1
SET NAME='saman'
WHERE clientNo='C001'
```

-- Insert values into multiple base tables via views

-- An important thing to remember is that SQL Server doesn't support inserting values into multiple base tables at once via views.

```
INSERT INTO mytestview1 (clientNo,name,city,Delivery_Address) VALUES ('C010','Aruni','Gampaha','Matara')
```

-- Insert into a single base table

```
INSERT INTO mytestview1 (clientNo,name,city) VALUES ('C010','Aruni','Gampaha')
```

4.6

```
CREATE VIEW view mytestview2
AS
SELECT clientNo, name, city, balance_due
FROM client
WHERE balance_due>12000

-- Then execute
DELETE from mytestview1_1 WHERE clientNo='C012'
```

4.7

```
CREATE VIEW mytestview3
AS
SELECT c.clientNo,c.name,c.city,s.Sales_Order_No,s.Sales_Order_Date,s.Delivery_Address
FROM client c, Sales_Order s
WHERE c.clientNo=s.ClientNo AND Sales_Order_Date>'01-May-2010'

-- Then execute
UPDATE mytestview3
SET Sales_Order_Date='23-Apr-2010'
WHERE clientNo='C007'

-- Drop the view and recreate it as follows
-- Note that the view is created using WITH CHECK OPTION
-- WITH CHECK OPTION prevents a view from updating or inserting (but not deleting.) rows that
are not visible through the view.
DROP VIEW mytestview3
GO

CREATE VIEW mytestview3
AS
SELECT c.clientNo,c.name,c.city,s.Sales_Order_No,s.Sales_Order_Date,s.Delivery_Address
FROM client c, Sales_Order s
WHERE c.clientNo=s.ClientNo AND Sales_Order_Date>'01-May-2010'
WITH CHECK OPTION

-- Then execute
UPDATE mytestview3
SET Sales_Order_Date='23-Apr-2010'
WHERE clientNo='C008'

UPDATE mytestview3
set Sales_Order_Date='23-May-2010'
WHERE clientNo='C008'
```